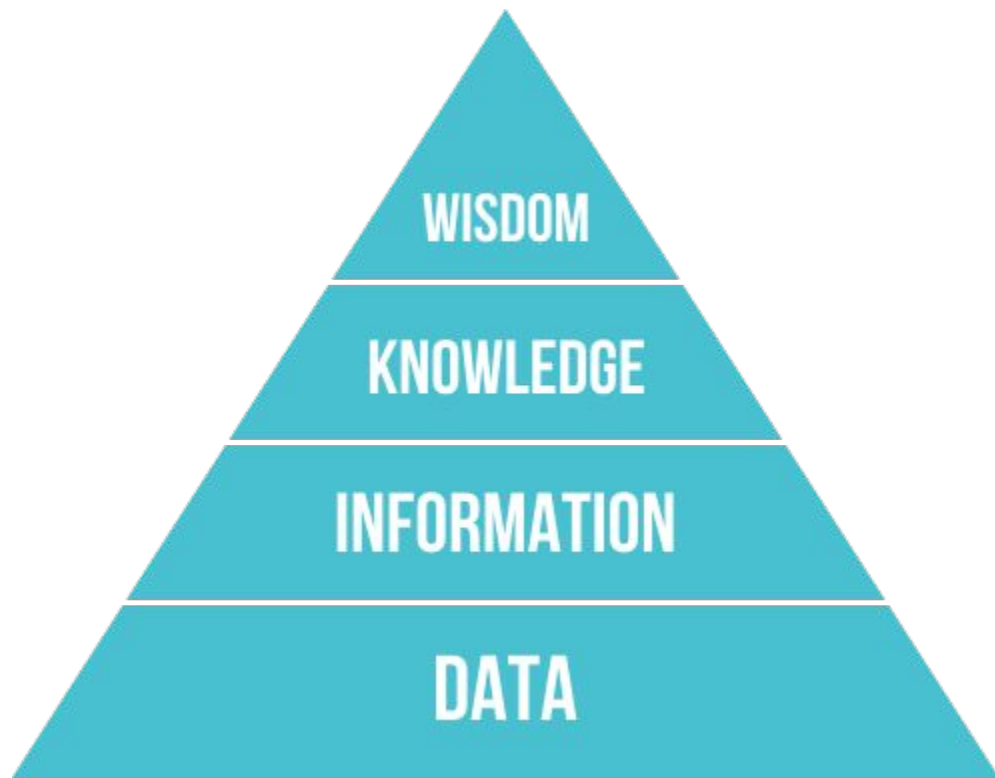


INTRODUCTION TO BIG DATA (UNIT 1)

15-July-2024

Dr. P. Rambabu, M. Tech., Ph.D., F.I.E.



Big Data Analytics and Applications

Defining Big Data

Big Data refers to extremely large datasets that are difficult to manage, process, and analyze using traditional data processing tools. The primary characteristics of Big Data are often described by the "**3 Vs**":

1. **Volume:** The amount of data generated is vast and continuously growing.
2. **Velocity:** The speed at which new data is generated and needs to be processed.
3. **Variety:** The different types of data (structured, semi-structured, and unstructured).

Additional characteristics sometimes included are:

4. **Veracity:** The quality and accuracy of the data.
5. **Value:** The potential insights and benefits derived from analyzing the data.

Big Data Types

Big Data can be categorized into **three main types**:

1. Structured Data: Organized in a fixed schema, usually in tabular form.

Examples include databases, spreadsheets.

2. Semi-structured Data: Does not conform to a rigid structure but contains tags or markers to separate data elements. Examples include JSON, XML files.

3. Unstructured Data: No predefined format or structure. Examples include text documents, images, videos, and social media posts.

Big Data Technologies

To manage and analyze Big Data, several technologies and tools are used, including:

- 1. Hadoop:** An open-source framework that allows for the distributed processing of large datasets across clusters of computers.
- 2. HDFS (Hadoop Distributed File System):** A scalable, fault-tolerant storage system.
- 3. MapReduce:** A programming model for processing large datasets with a distributed algorithm.
- 4. Spark:** An open-source unified analytics engine for large-scale data processing, known for its speed and ease of use.
- 5. NoSQL Databases:** Designed to handle large volumes of varied data. Examples include MongoDB, Cassandra, HBase.

Big Data Technologies

- 6. **Kafka:** A distributed streaming platform used for building real-time data pipelines and streaming applications.
- 7. **Hive:** A data warehousing tool built on top of Hadoop for querying and analyzing large datasets with SQL-like queries.
- 8. **Pig:** A high-level platform for creating MapReduce programs used with Hadoop

Examples of Big Data

Big Data is used in various industries and applications:

Healthcare: Analyzing patient data to improve treatment outcomes, predict epidemics, and reduce costs.

Finance: Detecting fraud, managing risk, and personalizing customer services.

Retail: Optimizing supply chain management, enhancing customer experience, and improving inventory management.

Telecommunications: Managing network traffic, improving customer service, and preventing churn.

Social Media: Analyzing user behavior, sentiment analysis, and targeted advertising.

The Evolution of Big Data Architecture



The architecture of Big Data systems has evolved to handle the growing complexity and demands of data processing. Key stages include:

Batch Processing: Initial systems focused on batch processing large volumes of data using tools like Hadoop and MapReduce. Data is processed in large chunks at scheduled intervals.

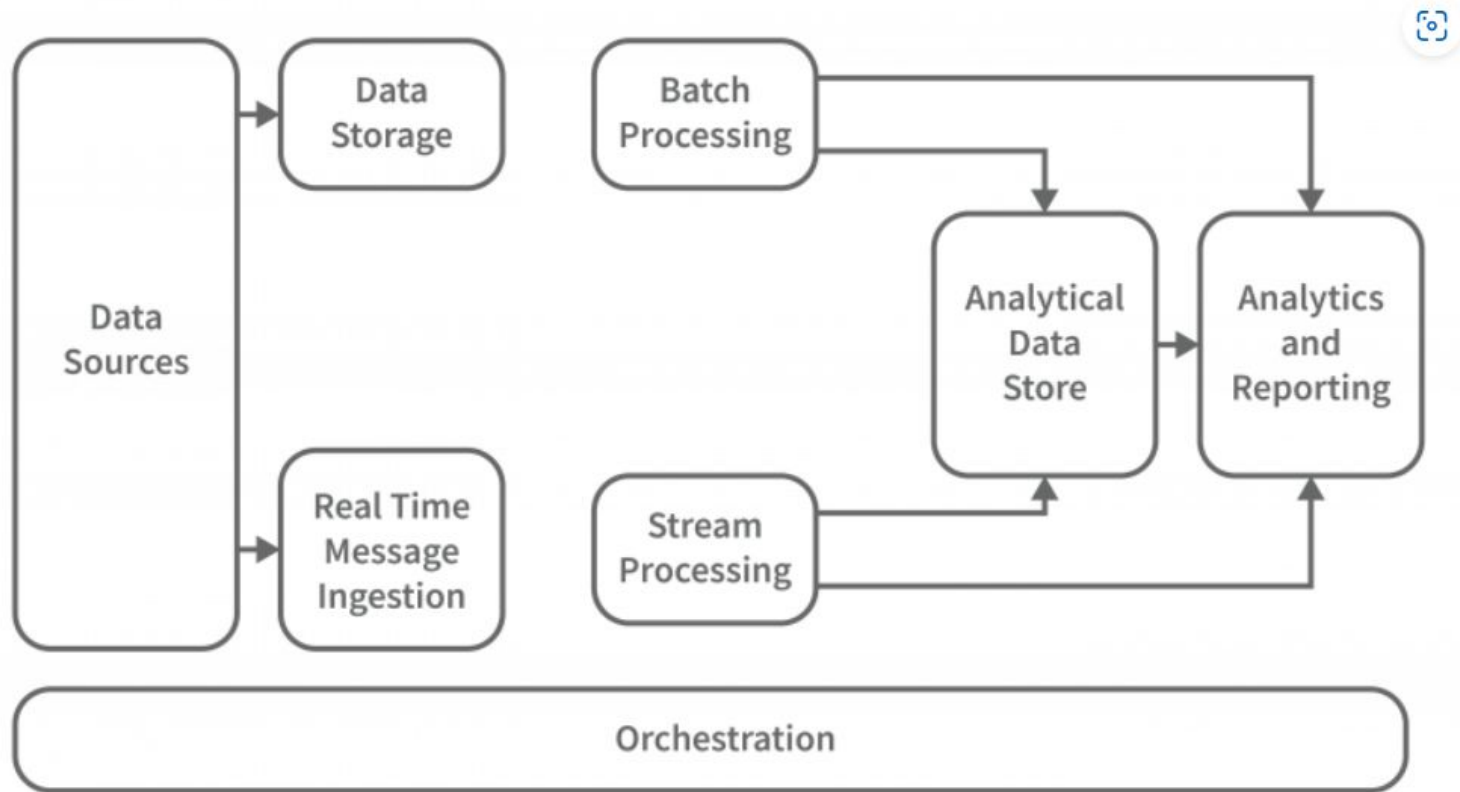
Real-time Processing: The need for real-time data analysis led to the development of technologies like Apache Storm and Apache Spark Streaming. These systems process data in real-time or near real-time.

The Evolution of Big Data Architecture

Lambda Architecture: A hybrid approach combining batch and real-time processing to provide comprehensive data analysis. The Lambda architecture consists of:

- ❑ **Batch Layer:** Stores all historical data and periodically processes it using batch processing.
- ❑ **Speed Layer:** Processes real-time data streams to provide immediate results.
- ❑ **Serving Layer:** Merges results from the batch and speed layers to deliver a unified view.

Kappa Architecture: Simplifies the Lambda architecture by using a single processing pipeline for both batch and real-time data, typically leveraging stream processing systems.



Evolution of Big Data Architecture

Evolution of Big Data and its ecosystem:

The evolution of Big Data and its ecosystem has undergone significant transformations over the years. Here's a brief overview:

Early 2000s:- Big Data emerges as a term to describe large, complex datasets.- Hadoop (2005) and MapReduce (2004) are developed to process large data sets.

2005-2010:- Hadoop becomes the foundation for Big Data processing.- NoSQL databases like Cassandra (2008), MongoDB (2009), and Couchbase (2010) emerge.- Data warehousing and business intelligence tools adapt to Big Data.

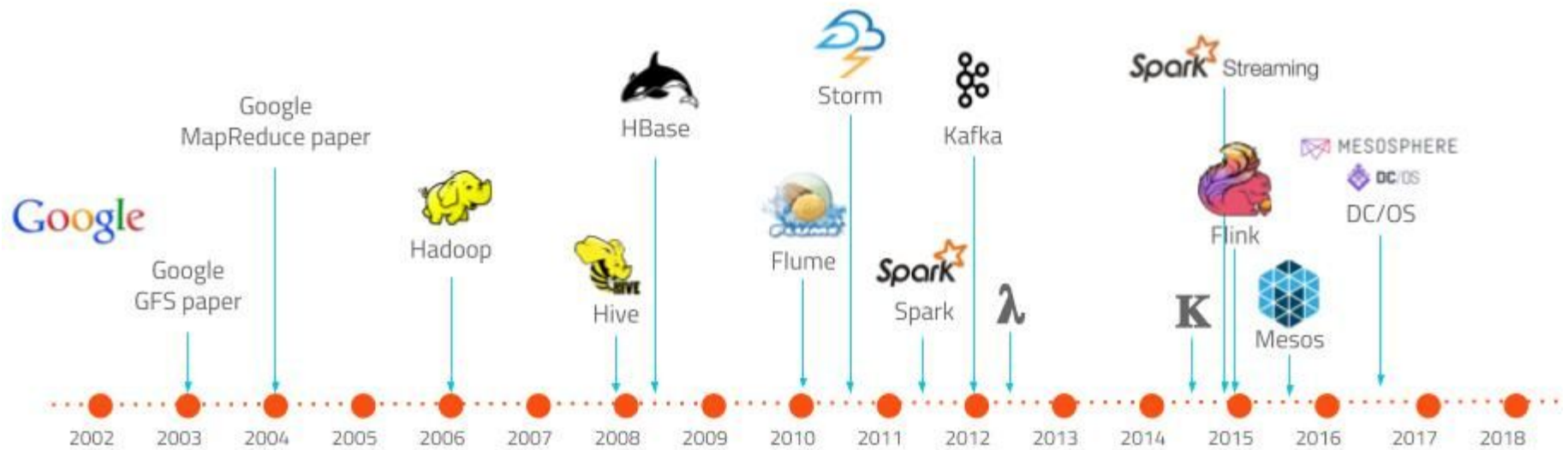
2010-2015:- Hadoop ecosystem expands with tools like Pig (2010), Hive (2010), and HBase (2010).- Spark (2010) and Flink (2011) emerge as in-memory processing engines.- Data science and machine learning gain prominence.

2015-2020:- Cloud-based Big Data services like AWS EMR (2012), Google Cloud Dataproc (2015), and Azure HDInsight (2013) become popular.- Containers and orchestration tools like Docker (2013) and Kubernetes (2014) simplify deployment.- Streaming data processing with Kafka (2011), Storm (2010), and Flink gains traction.

2020-present:- AI and machine learning continue to drive Big Data innovation.- Cloud-native architectures and serverless computing gain popularity.- Data governance, security, and ethics become increasingly important.- Emerging trends include edge computing, IoT, and Explainable AI (XAI).

The Big Data ecosystem has expanded to include:

1. Data ingestion tools e.g., [Flume](#) (2011), [NiFi](#) (2014)
2. Data processing frameworks e.g., [Hadoop](#) (2005), [Spark](#) (2010), [Flink](#) (2014)
3. NoSQL databases e.g., [Hbase](#) (2008), [Cassandra](#) (2008), [MongoDB](#) (2009), [Couchbase](#) (2011)
4. Data warehousing and BI tools e.g., [Hive](#) (2008), [Impala](#) (2012), [Tableau](#) (2003), [Presto](#) (2013), [SparkSQL](#) (2014), [Power BI](#) (2015)
5. Streaming data processing e.g., [Flink](#) (2010), [Kafka](#) (2011), [Storm](#) (2011), [Spark Streaming](#) (2013)
6. Machine learning and AI frameworks e.g., [Scikit-learn](#) (2010), [TensorFlow](#) (2015), [PyTorch](#) (2016)
7. Cloud-based Big Data services e.g., [AWS EMR](#) (2009), [Azure HDInsight](#) (2013), [Google Cloud Dataproc](#) (2016)
8. Containers and orchestration tools e.g., [Docker](#) (2013), [Kubernetes](#) (2014)

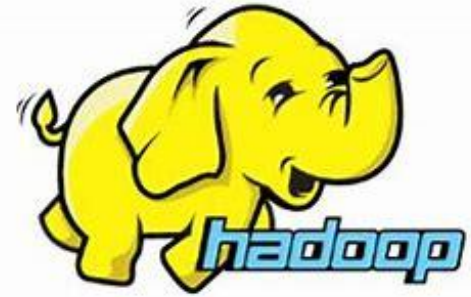


Evolution of Big Data and its ecosystem

Basics of Hadoop

Hadoop is an open-source framework designed for processing, storing, and analyzing large datasets in a distributed computing environment. It is widely used in the field of big data due to its scalability, fault tolerance, and flexibility.

Hadoop enables the analysis of big data through its distributed storage and processing capabilities. By breaking down large datasets into smaller chunks and processing them in parallel, Hadoop can handle data that is too large for traditional systems.



History of Hadoop

Apache Software Foundation is the developers of Hadoop, and its co-founders are Doug Cutting and Mike Cafarella. Its co-founder Doug Cutting named it on his son's toy elephant. In October 2003 the first paper release was Google File System. In January 2006, MapReduce development started on the Apache Nutch which consisted of around 6000 lines coding for it and around 5000 lines coding for HDFS. In **April 2006** Hadoop 0.1.0 was released.

The Hadoop framework allows for the distributed processing of large data sets across clusters of computers using simple programming models. It is designed to scale up from single servers to thousands of machines, each offering local computation and storage. It is used by many organizations, including **Yahoo**, **Facebook**, and **IBM**, for a variety of purposes such as data warehousing, log processing, and research.

Hadoop Architecture

Hadoop's architecture is based on a distributed computing model, which divides the workload across multiple nodes in a cluster. The key components of Hadoop architecture include:

1. HDFS (Hadoop Distributed File System): HDFS is the storage layer of Hadoop, designed to store large volumes of data across multiple machines. It splits the data into blocks and distributes them across the cluster, ensuring fault tolerance by replicating the blocks.

2. MapReduce: MapReduce is the processing layer of Hadoop. It is a programming model that allows for distributed processing of large datasets. The model consists of two main functions:

Map: Processes input data and produces intermediate key-value pairs.

Reduce: Aggregates the intermediate data and produces the final output.

3. YARN (Yet Another Resource Negotiator): YARN is the resource management layer of Hadoop. It manages and schedules the resources for various applications running in the cluster, ensuring efficient utilization of resources.

Hadoop Architecture and Ecosystem



Apache
HBase

Apache
Spark

Apache
Hive

Apache
Pig

Apache
Sqoop

Apache
Flume

MapReduce

Other Processing Jobs

YARN

(Yet Another Resource Navigator)

HDFS

(Hadoop Distributed File System)

Main Components of Hadoop Framework

Hadoop's ecosystem includes various tools and components that extend its functionality:

HDFS: As mentioned, HDFS is responsible for storing large datasets reliably across multiple nodes.

MapReduce: The core computational engine that processes data in parallel across the nodes.

Hive: A data warehouse infrastructure built on top of Hadoop that provides data summarization, query, and analysis.

Pig: A high-level scripting language that simplifies the process of writing MapReduce programs.

HBase: A distributed, scalable, NoSQL database that runs on top of HDFS.

Sqoop: A tool for efficiently transferring bulk data between Hadoop and structured data stores like relational databases.

Flume: A service for collecting, aggregating, and moving large amounts of log data from various sources to HDFS.

Hadoop Clustering

Hadoop clustering involves the use of multiple computers, known as **nodes**, to work together to store, process, and analyze large amounts of data. The cluster architecture is designed to provide high availability, fault tolerance, and scalability for handling big data workloads.

Key Components of Hadoop Clustering

Hadoop Distributed File System (HDFS):

NameNode: The master node that manages the metadata and namespace of the HDFS. It keeps track of the files, directories, and where the data blocks are stored across the cluster.

DataNode: The worker nodes that store and retrieve data blocks as instructed by the NameNode. Each DataNode periodically sends a heartbeat and block report to the NameNode to ensure its availability and status.

YARN (Yet Another Resource Negotiator):

ResourceManager: The master node that manages resources and schedules tasks across the cluster.

NodeManager: The worker nodes that manage resources on individual nodes, monitor resource usage, and report to the ResourceManager.

Types of Nodes in a Hadoop Cluster

Master Nodes:

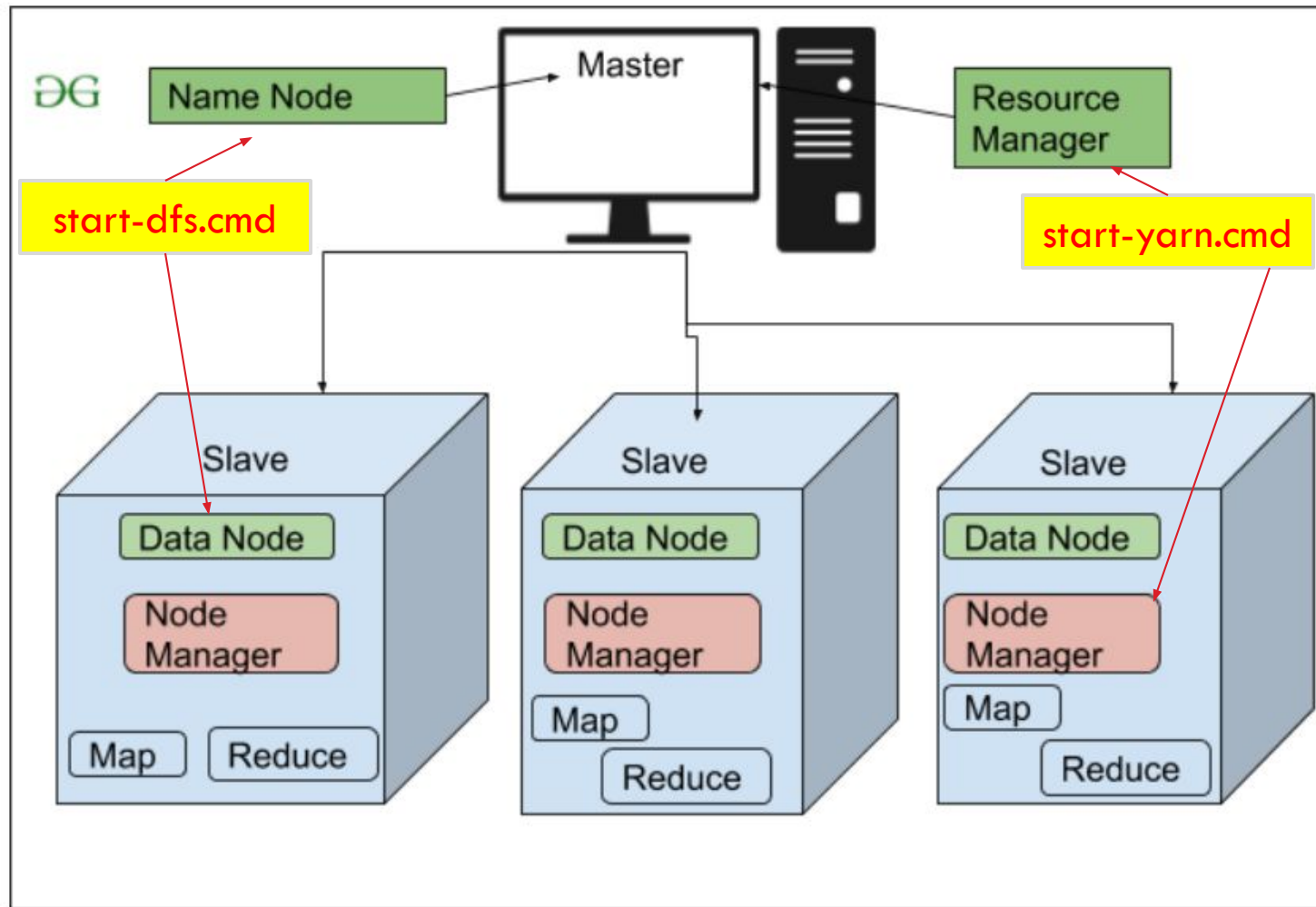
Typically, the **NameNode** and **ResourceManager** run on dedicated master nodes. These nodes manage the metadata and resource allocation for the cluster.

Worker Nodes:

These nodes run **DataNode** and **NodeManager** services. They are responsible for storing data and executing computational tasks.

Edge Nodes:

These are gateway nodes used for running client applications and tools to interact with the Hadoop cluster. They often host tools like Hive, Pig, and Sqoop.



Cluster Architecture

Single-Node Cluster:

All Hadoop services run on a single machine. This setup is primarily used for development, testing, and learning purposes.

Multi-Node Cluster:

A production environment where Hadoop services are distributed across multiple machines. Typically, one or more master nodes manage the cluster, and numerous worker nodes handle data storage and processing.

Data Processing in Hadoop Cluster

Data Ingestion:

Data can be ingested into HDFS using tools like **Apache Sqoop** (for structured data from relational databases) and **Apache Flume** (for streaming data).

Data Storage:

Data is stored in HDFS, which splits files into blocks and distributes them across multiple DataNodes for reliability and redundancy.

Data Processing:

Hadoop uses the **MapReduce** programming model for processing large datasets. YARN allocates resources and schedules the execution of MapReduce jobs across the cluster.

Advantages of Hadoop Clustering

Scalability:

Hadoop clusters can be scaled horizontally by adding more nodes to handle increased data volume and processing demands.

Fault Tolerance:

HDFS replicates data blocks across multiple nodes, ensuring data availability and resilience against node failures.

Cost-Effectiveness:

Hadoop clusters can be built using commodity hardware, making it a cost-effective solution for big data storage and processing.

Flexibility:

Hadoop supports various data formats (structured, semi-structured, and unstructured) and integrates with numerous data processing tools and frameworks.

Typical Workflow in a Hadoop Cluster

Data Loading:

Data is loaded into HDFS using tools like **Sqoop**, **Flume**, or **directly using HDFS commands**.

Data Storage:

The loaded data is split into blocks and stored across DataNodes, with replication for fault tolerance.

Resource Allocation:

YARN allocates resources based on the needs of the job and the availability of cluster resources.

Data Processing:

MapReduce or other processing frameworks (like **Apache Spark**) process the data. Jobs are broken into tasks, which are executed across the worker nodes.

Data Retrieval:

Processed data can be retrieved from HDFS and used for analysis, reporting, or further processing.

JAR file

A **JAR (Java ARchive)** file is a package file format typically used to aggregate many Java class files, associated metadata, and resources (text, images, etc.) into one file for distribution. The JAR format is based on the ZIP file format and is used for compressing and archiving purposes.

Why Use JAR Files?

Portability: JAR files enable Java applications to be packaged in a single file, making them easy to transport and distribute.

Efficiency: Combining multiple files into a single JAR file can improve download and load times.

Security: JAR files can be digitally signed, ensuring the integrity and authenticity of the files they contain.

Sealing: Packages within a JAR file can be optionally sealed, which means that all classes defined in that package must be found in the same JAR file.

Versioning: JAR files can include metadata about the version of the software they contain.

Creating a JAR File

To create a JAR file, you typically use the `jar` tool that comes with the JDK (Java Development Kit). Here's a step-by-step guide:

1. Create HelloWorld.java:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

2. Compile the Java program:

```
javac HelloWorld.java
```

3. Create a manifest file (manifest.txt):

Main-Class: HelloWorld

4. Create the JAR file:

```
jar cfm HelloWorld.jar manifest.txt HelloWorld.class
```

5. Run the JAR file:

```
java -jar HelloWorld.jar
```

This will output:

Hello, World!

Explanation of options **cfm**:

c - create a new archive.

f - specify the archive file name.

m - include a manifest file.

View the contents of a JAR file: You can list the contents of a JAR file using the **t** option.

```
jar tf MyJarFile.jar
```

Extract files from a JAR file: You can extract the contents of a JAR file using the **x** option.

```
jar xf MyJarFile.jar
```

Dr. Rambabu Palaka

Professor

School of Engineering

Malla Reddy University, Hyderabad

Mobile: +91-9652665840

Email: drrambabu@mallareddyuniversity.ac.in